ARTS COLLEGE, DHARMAPURI

Department of B.Sc. Computer Science

AI StudyBuddy API

AI assistant for smarter learning
and study support

PROJECT REPORT | 2026

SmartBridge Team	SWTID-2026-3504
Team leader	JAYAPRAKASH R
Team member	LOGITH M
Team member	MAYILSAMY S
Team member	GIRITHARAN V
Team member	SUGESH M

Node.js • Express.js • MongoDB • Google Gemini

Index

Section	Page
Cover page	1
Index	2
Project description and abstract	3
Scenario-based case study	4
Problem, objectives and scope	5
Software and hardware requirements	6
Epic 1: Project architecture	7
User roles and application workflow	8
Database design and relationships	9
Epic 2: Setup and configuration	10
Epic 3: Backend development	11
Authentication: Recorded execution	12
Study-material management	13
AI content-generation modules	14
API reference: Authentication and materials	15
API reference: AI and administration	16
Epic 4: Database configuration	17
Epic 5: Recorded execution outputs	18
Limitations and future enhancements	19
Conclusion and references	20

Project description and abstract

Project description

AI StudyBuddy API is a backend application that converts student study materials into useful revision resources. A registered user uploads a study file, selects an AI operation, and receives a summary, flashcards, a multiple-choice quiz, or a study plan. The application stores the material and its generated resources in MongoDB so that they can be retrieved later.

The project uses Node.js and Express.js to expose REST API endpoints. Mongoose defines the database models, JSON Web Tokens identify authenticated users, and bcryptjs hashes passwords. A dedicated utility communicates with Google Gemini through the Google Generative AI SDK. Multer handles incoming file uploads.

Abstract

Preparing revision material manually can take substantial effort. Students often read the same notes repeatedly while separately preparing questions and schedules. This project brings these activities into one backend service. Its modular structure separates routes, controllers, models, middleware, and external-service communication, making each responsibility easier to understand and maintain.

The system supports student and administrator roles. Students can manage their own learning materials, while administrative endpoints provide user listing, user deletion, and basic usage counts. Generated summaries, flashcards, quizzes, and study plans are embedded in a material document rather than stored in separate collections.

Report basis

This report follows the supplied LearnMate / AI StudyBuddy reference structure and describes the submitted GitHub source. The repository provides a backend API; a React dashboard is not included. Typed request examples remain explanatory; the captioned screenshots show recorded execution evidence. Execution screenshots were extracted from two team-supplied recordings and are included on pages 12 and 18. Sensitive values are masked. The videos provide evidence for the operations visible in those frames, not every project feature.

Scenario-based case study

Background

Consider a college student preparing for examinations in several subjects. The student has digital notes but little time to turn them into short revision material. Important definitions, examples, and topics are mixed together in long files. Repeatedly reading all the content makes it difficult to decide what to revise next.

Problem

The student needs a practical way to summarize a chapter, practise recall, answer questions, and divide revision across the available days. Completing each task manually creates repeated work. Keeping the resulting resources in separate tools also makes later revision less convenient.

Solution using AI StudyBuddy API

The student registers, logs in, and receives access credentials. A plain-text study file is uploaded with a descriptive title. The upload response supplies a material identifier, which is used for later AI requests. A summary request produces concise revision points; flashcard and quiz requests produce structured question sets.

For a study plan, the student sends a goal, the number of study days, and the hours available per day. These values are incorporated into the prompt sent to Gemini. The resulting schedule is stored with the same material, allowing the student to retrieve the notes and revision resources together.

Illustrative outcome

This workflow can reduce repeated preparation work and encourage active recall. The student still checks the generated facts against the original notes. The case study illustrates intended use; no measured improvement in marks, time savings, or learning outcomes is claimed.

Administrator usage

An administrator can inspect the registered user list, view student and material counts, and remove a user account with its related material records. The current implementation does not include a graphical administration dashboard or detailed AI-usage analytics.

Problem, objectives and scope

Problem statement

Students need several kinds of revision resources from the same study content. Creating separate summaries, question sets, and schedules introduces repetitive work. A reusable API can provide these functions through a consistent request format and preserve the generated material for later access.

Project objectives

Objective	Implemented approach
User access	Registration, login, access tokens and refresh tokens.
Material handling	Upload, list, retrieve and delete study materials.
Learning support	Generate summaries, flashcards, quizzes and study plans.
Persistence	Store users and materials through Mongoose models.
Administration	List users, delete accounts and return basic counts.
Maintainability	Separate routing, business logic and utility modules.

Scope of the submitted project

The deliverable is a REST backend intended for use through an API client or a separately developed frontend. The code includes two database models, three route groups, authentication middleware, upload middleware, and a Gemini integration utility. It returns JSON responses for application functions.

Scope boundaries

A browser interface, automated grading, progress tracking, payment management, and a deployed production service are not included in the inspected repository. Although the upload filter accepts PDF extensions, the controller reads files as UTF-8 text and does not implement PDF text extraction. Plain-text and Markdown files are the appropriate inputs for the current processing path.

Software and hardware requirements

Software environment

Component	Purpose in this project
Node.js and npm	Run JavaScript on the server and install dependencies.
Express.js	Provide routes, middleware and HTTP responses.
MongoDB and Mongoose	Persist records and apply document schemas.
Google Gemini access	Generate educational text from material content.
Postman or equivalent	Send requests, manage tokens and inspect responses.
Visual Studio Code	Read and maintain project files.
Windows, Linux or macOS	Host a compatible Node.js development environment.

Key dependencies declared in package.json

The repository declares Express 4.x, Mongoose 9.x, @google/generative-ai 0.24.x, bcryptjs 3.x, jsonwebtoken 9.x, Multer 2.x, cors, dotenv and express-async-errors. Nodemon is a development dependency. These are declared version ranges; installed versions depend on the lockfile and installation process.

Hardware planning

A modern laptop with sufficient memory for the editor, Node.js, an API client, and a browser is suitable for development. Around 8 GB RAM is a practical planning target, not a measured minimum for this application. Local MongoDB requires additional resources; a hosted database can reduce local setup needs.

Connectivity and configuration

AI generation requires internet access and a valid Gemini API key. Database connectivity is required before the server begins listening. Use a currently supported Node.js release compatible with the installed dependencies; do not rely on the older runtime minimum shown in the demonstration document.

Epic 1: Project architecture

Architecture overview

AI StudyBuddy follows a layered REST architecture. Express receives client requests and directs them through the relevant middleware and router. Controllers coordinate database operations and AI generation. Responses return as JSON, allowing the backend to serve different kinds of clients.

Layer	Responsibility and source
Client	Postman or a separately implemented frontend sends HTTP requests.
Entry point	index.js configures middleware, routes, errors and startup.
Route layer	auth.js, materials.js and admin.js map URLs to controllers.
Middleware	auth.js validates tokens; upload.js processes files.
Controller layer	Coordinates users, material records and AI operations.
Model layer	User.js and Material.js define MongoDB documents.
External service	gemini.js requests output from Gemini 2.5 Flash.

Request and data flow

Client → **Express route** → **middleware** → **controller.** A controller reads or writes MongoDB through a model. For AI generation, it first loads the material, sends a prompt through the Gemini utility, saves the generated result, and returns a JSON response to the client.

MVC interpretation

The models represent stored data, while controllers implement request handling. Because the project is an API, its presentation boundary consists of JSON responses rather than server-rendered HTML views. Routes connect the incoming request to the appropriate operation. This separation makes the project easier to explain, extend, and debug.

User roles and application workflow

Student workflow

Step	Action and result
1. Register	Submit name, email and password to create a user.
2. Authenticate	Log in and retain the returned access and refresh tokens.
3. Upload	Send a file and optional title; retain the returned material ID.
4. Generate	Call an AI endpoint using that material ID.
5. Review	Check the generated content for accuracy and relevance.
6. Retrieve	Load the material again to see stored learning resources.
7. End session	Call logout and discard tokens in the client.

Administrator workflow

Administrative routes apply both the authentication middleware and an admin-role check. A permitted caller can retrieve users, delete a user and the associated materials, or obtain basic totals. The statistics response names its student count totalUsers, although the query specifically counts records with role equal to student.

Access-control behavior

Students receive only their own materials in the material list. Single-material retrieval and deletion compare the stored user reference with the authenticated user ID. Administrators are exempt from these ownership checks. AI generation handlers currently load the material by ID without repeating the ownership check; this is a limitation described on page 19.

Session renewal

When an access token expires, the client can send the refresh token in the x-refresh-token header. A valid token produces a new token pair. Logout returns a confirmation message; it does not maintain a server-side token blocklist.

Database design and relationships

Actual entity relationship

User (one) → Material (many). Each material stores a required ObjectId reference to a user. Summaries, flashcards, quizzes and the study plan are fields within the Material document. They are not independent collections in the submitted code.

User field	Definition
_id	MongoDB document identifier.
name	Required string, trimmed.
email	Required unique string, converted to lowercase.
password	Required string; hashed before storage.
role	student or admin; default is student.
createdAt / updatedAt	Managed through Mongoose timestamps.

Material field	Definition
user	Required ObjectId reference to User.
title / content	Required strings for label and source text.
filename	Original uploaded filename.
summary / studyPlan	Generated text strings.
flashcards	Array of question-and-answer objects.
quiz	Array of question, options and answer objects.
createdAt / updatedAt	Managed through Mongoose timestamps.

Design implications

A material record brings source text and generated resources together. A new generation replaces the relevant stored field rather than preserving earlier versions. Deleting a material removes its embedded AI content. User deletion explicitly removes the related material records through a separate query.

Epic 2: Setup and configuration

Local setup procedure

Clone the repository or extract the downloaded source, open its folder in the editor, and install the dependencies. Create a local .env file using the variable names read by the application. Keep actual credentials outside the report and source-control history.

```
npm install
node index.js
```

The inspected package.json uses nodemon server.js for npm start, while the repository entry point is index.js. Run node index.js directly, or change the start script to nodemon index.js before using npm start. This report does not modify the submitted repository.

Environment variable template

```
PORT=5000
MONGO_URI=<your MongoDB connection string>
JWT_ACCESS_SECRET=<your access-token secret>
JWT_REFRESH_SECRET=<your refresh-token secret>
GEMINI_API_KEY=<your Gemini API key>
CLIENT_URL=http://localhost:3000
```

Configuration responsibilities

MONGO_URI selects the database connection. Separate access and refresh secrets are used by token creation and verification. CLIENT_URL controls the permitted CORS origin, with localhost:3000 as the fallback. PORT determines the listening port, defaulting to 5000.

Startup sequence

The application loads environment variables, ensures that the uploads folder exists, enables JSON parsing and CORS, mounts route groups, and registers the error handler. It then connects to MongoDB. Only after the database connection succeeds does the HTTP server begin listening.

Epic 3: Backend development

Project structure

```
index.js
package.json
src/
  controllers/
    authController.js
    materialController.js
    adminController.js
  middleware/
    auth.js
    upload.js
  models/
    User.js
    Material.js
  routes/
    auth.js
    materials.js
    admin.js
  utils/
    db.js
    gemini.js
    tokens.js
```

Core system implementation

index.js creates the Express application, installs middleware, exposes the root health response, and mounts the authentication, material, and administration routes. express-async-errors allows rejected asynchronous handlers to reach the Express error handler. The handler uses err.status when available and otherwise returns status 500.

Controllers perform application operations rather than placing all logic in route files. Authentication handles account and token functions. Material operations coordinate file input, database access, and prompt creation. Administration provides account management and statistics.

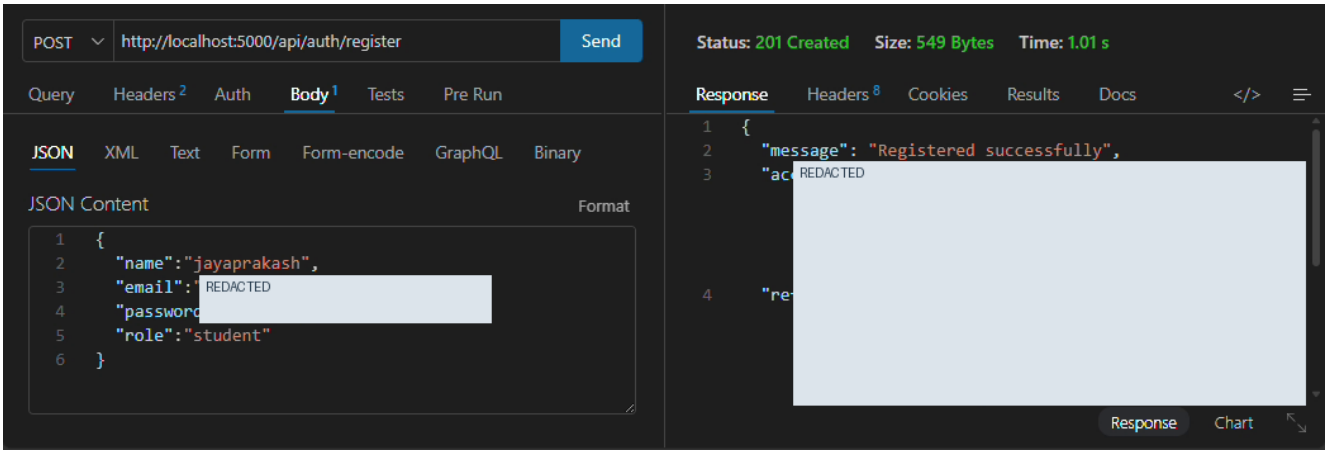
Utility boundaries

db.js owns the database connection, tokens.js owns token creation, and gemini.js owns the AI model call. These small boundaries allow future changes to one responsibility without rewriting every route. They also provide clear locations for additional validation, logging, and automated tests.

Authentication: Recorded execution

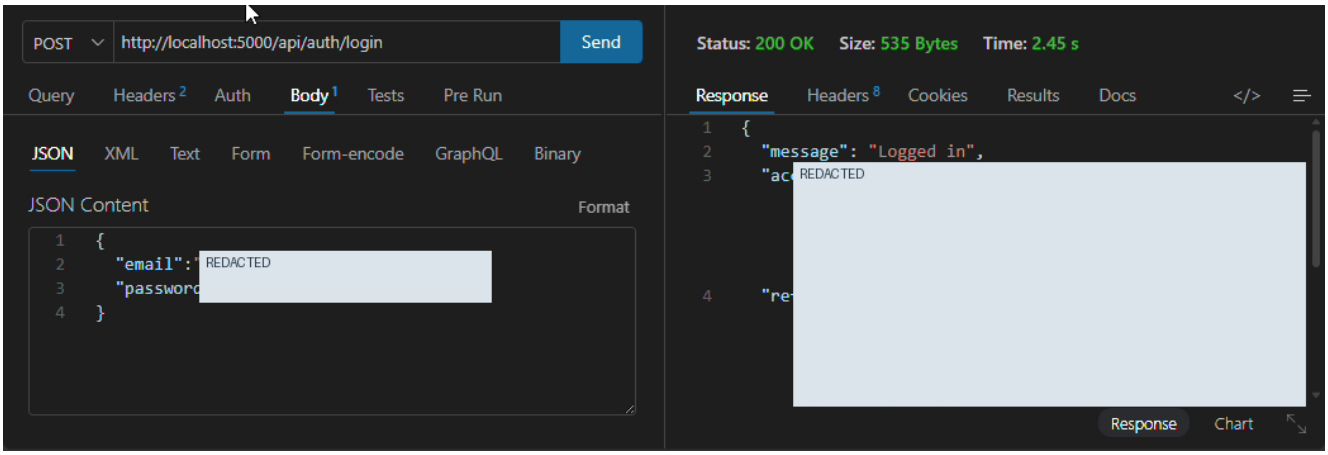
Thunder Client in Visual Studio Code shows successful registration and login. These screenshots are taken from Recording study buddy 1.mp4. Request credentials and issued tokens are masked; endpoint URLs, status codes and response messages remain visible.

Figure 1. User registration - 201 Created



Video 1, 01:05. POST /api/auth/register returns Registered successfully. The request shows the student role.

Figure 2. User login - 200 OK



Video 1, 01:10. POST /api/auth/login returns Logged in. Access and refresh token values are hidden.

Authentication behavior

The source uses bcrypt password hashing and JWTs. Access tokens expire in 15 minutes and refresh tokens in 7 days. Protected routes require a Bearer access token; refresh requests use the x-refresh-token header. The screenshots confirm the displayed success responses, but do not verify all failure cases or token-expiry behavior.

Study-material management

Upload processing

The upload endpoint expects multipart/form-data with a file field named file. A title can be supplied in the same request. Multer writes the upload to a temporary uploads directory and generates a unique disk filename. The controller reads the file, creates a Material record, removes the temporary file, and returns status 201.

Input or rule	Current behavior
File field	file; one uploaded file per request.
Optional title	Falls back to the original filename.
Accepted extensions	.txt, .md and .pdf.
Size limit	5 × 1024 × 1024 bytes.
Text processing	Reads the saved file as UTF-8.
Stored data	User reference, title, content and original filename.

List, retrieve and delete

The list endpoint orders materials from newest to oldest. It excludes content, flashcards, quiz, and studyPlan from the list response, reducing the amount returned. The summary field is not excluded. Students are filtered by their user ID, while administrators can list all materials.

The single-record endpoint returns a material after checking existence and ownership. The delete endpoint applies the same ownership rule and removes the record. Missing records receive status 404, while an unauthorized student receives status 403.

File-format limitation

An allowed extension does not guarantee that the content is processed correctly. The current UTF-8 reader does not extract text from PDF structures. Use .txt or .md for the demonstrated workflow. A future PDF parser should extract usable text before the material is saved or sent to the AI service.

AI content-generation modules

Gemini service integration

The Gemini utility creates a GoogleGenerativeAI client with GEMINI_API_KEY and selects gemini-2.5-flash. Its askGemini function submits a prompt and returns the response text. Material controllers decide what prompt to send and how to store the result.

Feature	Prompt and stored output
Summary	Requests clear, concise bullet points; stores a summary string.
Flashcards	Requests question/answer JSON; stores a flashcards array.
Quiz	Requests multiple-choice JSON with options and answer; stores a quiz array.
Study plan	Requests a day-by-day schedule; stores a studyPlan string.

Generation options

Flashcard and quiz endpoints read count from the request body and use 5 when a truthy value is not provided. The study-plan endpoint reads goal, hoursPerDay and days. Its fallbacks are a general understanding-and-retention goal, 2 hours per day, and a 7-day schedule.

Parsing and persistence

Flashcard and quiz handlers remove Markdown code-fence markers from the model response and call JSON.parse. The parsed result is assigned to the material and saved. Summary and study-plan outputs remain text strings. Successful requests return the newly generated field in a JSON response.

Quality and failure handling

AI output varies and should be checked before use in examination preparation. Invalid JSON, external-service errors, or persistence failures can interrupt a request. The current implementation has no structured retry policy or strict AI-output validator. Generating the same resource again overwrites that field in the material record.

API reference: Authentication and materials

Local base URL: **http://localhost:5000**, unless PORT is changed. JSON requests use Content-Type: application/json. Protected requests include Authorization: Bearer <accessToken>.

Method and path	Purpose
POST /api/auth/register	Create an account and return tokens.
POST /api/auth/login	Validate credentials and return tokens.
POST /api/auth/refresh	Renew tokens using x-refresh-token.
POST /api/auth/logout	Return a logout confirmation.
POST /api/materials/upload	Upload multipart file and optional title.
GET /api/materials	List permitted material records.
GET /api/materials/:id	Retrieve a permitted material.
DELETE /api/materials/:id	Delete a permitted material.

Illustrative registration body

```
{
  "name": "Sample Student",
  "email": "student@example.com",
  "password": "<sample password>"
}
```

Illustrative response shape

```
{
  "message": "Registered successfully",
  "accessToken": "<issued access token>",
  "refreshToken": "<issued refresh token>",
  "user": {"id": "<user id>",
           "name": "Sample Student", "role": "student"}
}
```

The placeholders above show the response structure from the controller; they are not captured credentials or a live execution result.

API reference: AI and administration

Method and path	Body or purpose
POST /api/materials/:id/summarize	No feature-specific body required.
POST /api/materials/:id/flashcards	Optional count value.
POST /api/materials/:id/quiz	Optional count value.
POST /api/materials/:id/study-plan	goal, hoursPerDay and days.
GET /api/admin/users	List users without password fields.
DELETE /api/admin/users/:id	Delete a user and related materials.
GET /api/admin/stats	Return student and material counts.

Illustrative AI requests

```
POST /api/materials/<material id>/quiz
Authorization: Bearer <access token>
Content-Type: application/json

{"count": 5}

POST /api/materials/<material id>/study-plan
Authorization: Bearer <access token>
Content-Type: application/json

{
  "goal": "Revise database fundamentals",
  "hoursPerDay": 2,
  "days": 7
}
```

Response keys

The AI endpoints respectively return summary, flashcards, quiz, and studyPlan. The admin statistics endpoint returns totalUsers and totalMaterials. Administrative requests must pass both token verification and the role check. Deleting a user affects stored data and should be tested only with a disposable account.

Epic 4: Database configuration

Connecting MongoDB

The database utility loads configuration and passes MONGO_URI to mongoose.connect. A successful connection is logged, after which the entry point starts the HTTP listener. The connection string can refer to a local database or an appropriately configured hosted MongoDB service.

Data lifecycle

Operation	Database action
Registration	Find an existing email; create a User if permitted.
Material upload	Create a Material linked to req.user.userId.
AI generation	Load a Material and save the generated field.
Material listing	Filter by user for students; sort newest first.
Material deletion	Delete the selected material document.
User deletion	Delete the user, then delete matching materials.
Statistics	Count student users and all materials.

Validation and relationships

Mongoose requires the main identity and content fields and constrains the role to student or admin. The user email has a unique index declaration. The material user reference expresses a relationship but does not automatically perform cascading deletes; the administrator controller supplies that cleanup logic.

Consistency considerations

User deletion and material deletion run as separate operations. Without a transaction, an interruption between them could leave inconsistent data. Similarly, AI generation requires an external request followed by a database save; a generated response is not persisted until the save succeeds.

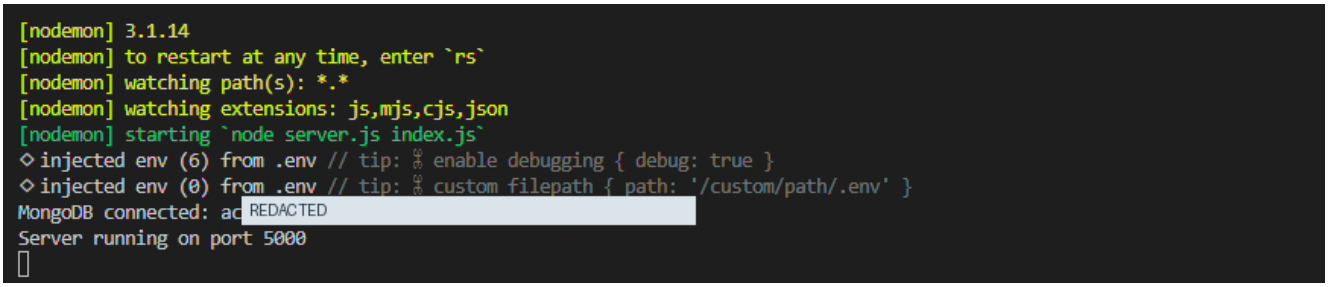
Operational checks

During local verification, confirm the intended database is selected, a sample user is stored with a password hash, and an uploaded material refers to that user. After generation, inspect the relevant embedded field. Do not include database credentials, password hashes, or tokens in submission screenshots.

Epic 5: Recorded execution outputs

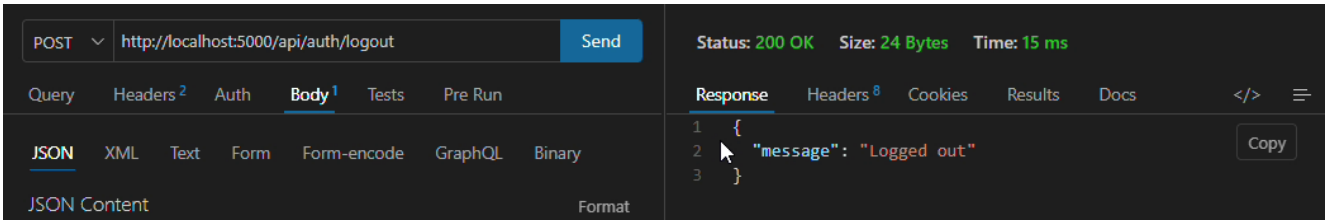
The following frames document startup and successful API responses in the supplied videos. Screenshots are cropped for readability, with sensitive fields masked.

Figure 3. Database connection and server startup



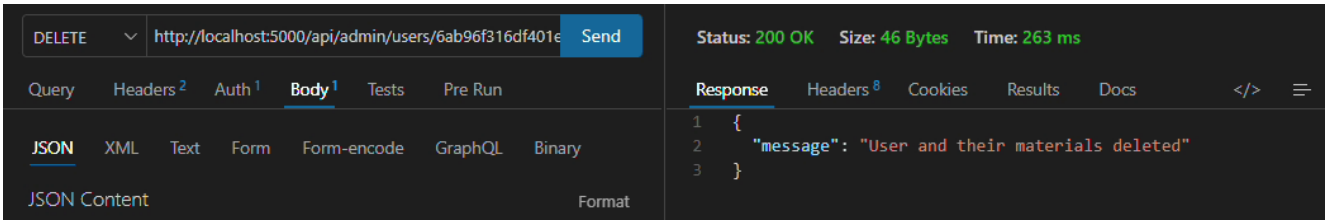
Video 1, 01:20. Terminal shows MongoDB connected and Server running on port 5000. The database host is masked.

Figure 4. Logout - 200 OK



Video 1, 01:20. POST /api/auth/logout returns Logged out. The current implementation expects the client to discard its tokens.

Figure 5. Administrative user deletion - 200 OK



Video 2, 00:02. DELETE /api/admin/users/:id returns User and their materials deleted. This is a recorded response; no deletion was performed while preparing the report.

Evidence scope: the recordings do not show material uploads, AI summary/quiz/flashcard/study-plan generation, or negative access tests. These operations still require separate execution evidence.

Limitations and future enhancements

Implementation findings

Current limitation	Recommended improvement
Start script points to server.js	Point it to index.js or run node index.js.
PDF read as plain UTF-8	Add real PDF text extraction and content validation.
Registration accepts caller role	Force public accounts to student; restrict admin creation.
AI handlers omit ownership checks	Check the material owner before all generation operations.
Refresh tokens are stateless	Add revocation, rotation tracking and user-state checks.
AI JSON is parsed directly	Validate output schema and handle parse failures clearly.
Limited input/error validation	Validate IDs, counts and study-plan inputs; map errors clearly.

Compatibility and reliability

The User pre-save hook contains a next() callback branch while the declared Mongoose major version is 9. Review that branch against the installed version and prefer a consistent async hook style. File cleanup on failed uploads and transaction-aware user deletion would also improve reliability.

Future functional scope

Useful extensions include a student web interface, generated-resource version history, quiz attempts and scores, progress tracking, document text extraction, and exportable revision notes. These are proposed enhancements, not features verified in the current repository.

Development priorities

First correct startup and authorization behavior. Next strengthen validation and test failure paths. After these foundations are checked, add user-facing features. This order supports a more dependable application without overstating the readiness of the current academic implementation.

Conclusion and references

Conclusion

AI StudyBuddy API demonstrates how a modular Node.js backend can connect account management, material storage, and generative AI. Its central workflow turns uploaded text into summaries, flashcards, quizzes, and a study schedule, with the results stored alongside the original material.

The project provides practical experience with Express routing, Mongoose schemas, JWT authentication, multipart uploads, external API calls, and structured JSON responses. Its separation of responsibilities creates a clear basis for further development and for explaining the system during a project demonstration.

The current source also has limitations that should be addressed before wider deployment. The report distinguishes implemented functionality from intended benefits and future enhancements. The supplied videos document registration, login, logout, startup and an administrative deletion response. AI-generation outputs and negative test cases require separate verification.

Primary project sources

1. Submitted repository:
github.com/jayaprakash161107-cpu/AI-Studybuddy-API

Source snapshot reviewed: commit 5b27395bf5f0b26b8a56c4d3d9608198692fbafc. Key files: index.js, package.json, src/routes, src/controllers, src/models, src/middleware and src/utils.

2. Supplied reference document:
LearnMate - AI assistant for smarter learning and study support.
Open the supplied demonstration document. Used for section structure; implementation details were checked against the repository.

Execution evidence sources

Recording study buddy 1.mp4 (86.13 seconds): frames at 01:05, 01:10 and 01:20. Recording 2.mp4 (4.70 seconds): frame at 00:02. Supplied by the project team; used for Figures 1-5.

Submission details

Prepared for SmartBridge Team SWTID-2026-3504, Arts College, Dharmapuri, Department of B.Sc. Computer Science. Team leader: JAYAPRAKASH R. Members: LOGITH M, MAYILSAMY S, GIRITHARAN V and SUGESH M. Report date: 28 September 2026.

DRIVE :

https://drive.google.com/file/d/1mApSb2EIX-GcrNCSNR1qeZOk8UngQgJt/view?usp=drive_link

GITHUB :

https://github.com/jayaprakash161107-cpu/AI-Studybuddy-API